comma (`,`) character to compose functions on the fly. For example, you could combine the `genreQuote` function with `logQuote` as follows:

```
$ echo 'FANTASY' | http :9102/genreQuote,logQuote
```

Since `logQuote` is a consumer, the HTTP response has a 202 status with no body. If you check the application logs, you'll see that the random quote by genre has been printed out instead.

Spring Cloud Function integrates with several communication channels. You have seen how you can expose functions through exchanges and queues leveraging Spring Cloud Stream and how to expose them as HTTP endpoints. The framework also supports RSocket, which is a binary reactive protocol, and CloudEvents, a specification standardizing the format and distribution of events in cloud architectures (https://cloudevents.io).

*CloudEvents* can be consumed over HTTP, messaging channels like AMPQ (RabbitMQ), and RSocket. They ensure a standard way of describing events, thus making them portable across a wide variety of technologies, including applications, messaging systems, build tools, and platforms.

Since Quote Function is already configured to expose functions as HTTP endpoints, you can make it consume CloudEvents without changing any code. Ensure the application is up and running, and then send an HTTP request with the additional headers defined by the CloudEvents specification:

```
$ echo 'FANTASY' | http :9102/genreQuote \
    ce-specversion:1.0 \          ❶
    ce-type:quote \               ❷
    ce-id:394                     ❸
```

❶ The CloudEvents specification version